

Ghana Emergency Response System

Complete Code Review & Architecture Guide

Version 2.0 Enhanced | EBRS (Emergency Broadcast & Response System)

Generated: June 29, 2026

1. Project Overview

The Ghana Emergency Response System (EBRS) is a full-stack emergency reporting and coordination platform. Citizens report fires, medical, or crime emergencies with GPS location, voice recordings, and media attachments. Administrators manage incidents through a real-time dashboard. Field responders track their location via a mobile Progressive Web App (PWA).

Technology Stack

- Frontend: Next.js 13, React 18, Leaflet maps, Socket.IO client
- Backend (Primary): Supabase (PostgreSQL, Auth, Storage, Realtime)
- Backend (Legacy): Express.js + MongoDB + Socket.IO (port 4000)
- Deployment: Netlify (HTTPS), optional Render for Express backend
- SMS: Twilio integration via backend API

2. Project Structure

```
emergency/
  frontend/      Main Next.js app (port 3000)
  pages/         Application routes
  components/    Maps, chat components
  lib/          Supabase, analytics, SMS, offline utilities
  public/       PWA manifests, service workers, icons
  backend/      Legacy Express API (port 4000)
  database/     Supabase SQL migrations and fixes
  scripts/      Utility scripts
```

3. Frontend Pages & Routes

Route	File	Purpose
/	index.js	Public emergency reporting (Fire/Medical/Crime)
/login	login.js	Admin authentication via Supabase
/admin	admin.js	Full admin dashboard (~3,500 lines)
/responder	responder.js	Field responder PWA with GPS tracking
/reports	reports.js	Report archive and history
/manage-admins	manage-admins.js	Admin user management
/location-test	location-test.js	GPS debugging page

Legacy pages (old Express backend): index-old-backend.js, admin-old-backend.js, user.js, reports-old-backend.js. These are kept for reference but are not the active production path.

4. Core Data Flow

Step 1 - Victim Reporting (/):

- User selects emergency type (Fire 192, Medical 193, Crime 191)
- Browser Geolocation API captures GPS coordinates
- Optional voice recording (MediaRecorder, 60s max)
- Optional photo/video upload
- Data inserted into Supabase 'reports' table
- Media files uploaded to 'emergency media' storage bucket

Step 2 - Admin Dashboard (/admin):

- Real-time subscription to report changes via Supabase Realtime
- Interactive Leaflet map with emergency markers
- Status management (pending, in_progress, resolved, etc.)
- Admin notes, activity logging, group chat
- Analytics, SMS alerts, nearby facilities lookup
- Live responder location tracking on map

Step 3 - Responder App (/responder):

- Login with responder ID (e.g., FIRE001)
- View assigned emergency details
- Start GPS tracking - streams position to responder_locations table
- Status updates: Available, En Route, On Scene
- Distance, ETA, and speed calculation
- PWA installable on mobile devices

5. Key Libraries (frontend/lib/)

Module	Purpose	
supabase.js	Client, file upload, CRUD, realtime subscriptions	
analytics.js	Response times, trends, CSV export	
activityLogger.js	Audit trail (login, status, notes)	
notifications.js	Browser push notifications	
offline.js	Service worker, offline queue	
sms.js	Twilio SMS alerts via backend	
nearbyFacilities.js	Hospitals, fire/police via Google Places	
featureFlags.js	Feature toggles (AI disabled by default)	

6. React Components

- EmergencyMap.js - Leaflet map for admin (report markers, responder tracking)
- ResponderMap.js - Responder navigation map with ETA and distance
- GroupChat.js - Real-time admin group chat via chat_messages table
- MapView.js - Legacy map component for old Express backend

7. Database Schema (Supabase)

Main PostgreSQL tables with Row Level Security (RLS) enabled:

- reports - Emergency incident records with GPS, media URLs, status
- admin_profiles - Role-based admin access control
- chat_messages - Admin group chat messages
- responder_locations - Live GPS coordinates from field responders
- responders - Responder accounts (ID-based login)
- evidence_files - Additional evidence attachments
- activity_logs - Audit trail of admin actions
- admin_notes - Notes attached to specific reports

Realtime subscriptions are enabled on key tables. RLS policies restrict data access based on authenticated user roles.

8. Legacy Backend (backend/)

Express + Socket.IO server providing:

- MongoDB or in-memory fallback for report storage
- Multer file uploads (images, video, audio up to 50MB)
- Twilio SMS routes (/api/sms)
- Google Directions API for route optimization

IMPORTANT: Production currently uses Supabase directly from the frontend. The Express backend is legacy; pages like user.js still reference localhost:4000.

9. PWA & Mobile Support

- Two manifests: manifest.json (public) and responder-manifest.json
- Service workers: sw.js and responder-sw.js
- _app.js selects manifest/SW based on current route
- HTTPS required for iPhone GPS (Safari blocks geolocation on HTTP)
- Clear cache functionality built into responder app

10. Completed Features (Production Ready)

Public Emergency Reporting

- Emergency type selection (Fire, Medical, Crime, Other)
- GPS location capture with accuracy display
- Voice recording (60 second maximum)
- Photo and video upload with previews
- Contact information and anonymous reporting
- Real-time submission to Supabase

Admin Dashboard

- Real-time emergency feed with auto-refresh
- Interactive map with emergency markers
- Status management and admin notes system
- Activity logging and group chat
- Dark mode, keyboard shortcuts, evidence gallery
- Responder tracking on map, analytics dashboard
- SMS alert configuration, nearby facilities lookup

Responder Mobile App

- Responder login (ID-based authentication)
- Emergency assignment display
- Real-time GPS tracking with live navigation map
- Status updates: Available, En Route, On Scene
- Distance, ETA, speed, and battery indicators
- PWA support (installable on home screen)

11. Optional / Incomplete Features

- AI emergency classification - Not implemented (requires OpenAI API key)
- Full mobile admin layout optimization - Partially done
- Evidence annotation tools - Not implemented
- Chat read receipts and delivery status - Not implemented
- Full offline operation queuing - Partially implemented

12. Known Issues

PWA Manifest Conflict (Low Impact)

When installing the responder app, it sometimes opens the main app. Workaround: use the clear cache button or fresh install. Browser version works correctly.

Browser Cache Persistence (Low Impact)

Old versions may be cached aggressively. Version badges and clear cache button are provided as workarounds.

13. Code Observations

- admin.js is very large (~3,500 lines) - consider splitting into hooks/components
- Dual architecture: Supabase primary, Express/MongoDB legacy remains
- Inline styles throughout - no CSS framework; uses globals.css and admin-mobile.css
- Storage bucket name 'emergency media' contains a space - must match Supabase exactly
- Feature flags in featureFlags.js centralize toggles; AI is off by default

14. Completion Status

Category	Status	Notes
Core Features	100%	All essential features complete
Optional Enhancements	40%	Nice-to-have features remain
Bug Fixes	95%	Critical bugs resolved
Production Readiness	Ready	Deploy as-is recommended

15. Local Development

Frontend:

```
cd frontend
npm install
npm run dev      # http://0.0.0.0:3000
```

Optional Legacy Backend:

```
cd backend
npm install
npm run dev      # http://localhost:4000
```

Required Environment Variables:

```
NEXT_PUBLIC_SUPABASE_URL
NEXT_PUBLIC_SUPABASE_ANON_KEY
```

16. Ghana Emergency Numbers

Type	Number	Service
Fire	192	Ghana Fire Service
Medical	193	National Ambulance Service
Crime	191	Ghana Police Service